CSI-270 Data Structures with Python

Final Project: The Partition Constraint Problem

Problem Statement

You are given a collection of integers and two constraint parameters ***k*** and ***l***. Your task is to create **partitions** (sub-collections) of these integers that satisfy specific mathematical constraints.

The Challenge

Given:

- A collection of integers: $[a_1, a_2, a_3, ..., a_n]$
- Two positive integers: ***k*** (minimum sum constraint) and ***l*** (minimum diversity constraint)

Create multiple partitions that **maximize the number of partitions** while satisfying:

1. **Sum Constraint**: Each partition must have a sum of at least ***k***
2. **Diversity Constraint**: Each partition must contain at least ***l*** non-empty indices
3. **Reconstruction Property**: When you sum all values at each index position across all partitions, you must get back the original collection

Optimization Goal: Among all valid solutions, find the one with the **maximum number of partitions**

What Does "Index" Mean Here?

Think of your original collection as having fixed positions (indices). Each partition is like a "layer" that contributes to these positions.

For example, if your original collection is [10, 5, 8]:

- Index 0 has value 10
- Index 1 has value 5
- Index 2 has value 8

Each partition also has these same three index positions. Some positions might be 0 (empty), and some might have positive values. The key rule is: **when you add up all the values at position i across all partitions, you must get the original value at position i**.

Mathematical Example

Example 1: Basic Case

Original Collection: [10, 5, 8]
Constraints: $k = 10, l = 2$

Valid Partition Solution:

Partition 1: [7, 3, 0] → Sum = $10 \geq k$ ✓ Non-empty indices: $2 \geq l$ ✓

Partition 2: [3, 2, 8] → Sum = $13 \geq k$ ✓ Non-empty indices: $3 \geq l$ ✓

Verification:

- Index 0: 7 + 3 = 10 ✓
- Index 1: 3 + 2 = 5 ✓
- Index 2: 0 + 8 = 8 ✓
- All original values reconstructed correctly!

Example 2: Multiple Partitions - Optimization Matters!

Original Collection: [15, 20, 10, 5]
Constraints: $k = 15, l = 2$

Solution A (2 partitions):

Partition 1: [15, 0, 0, 5] → Sum = $20 \geq 15$ ✓ Non-empty: $2 \geq 2$ ✓

Partition 2: [0, 20, 10, 0] → Sum = $30 \geq 15$ ✓ Non-empty: $2 \geq 2$ ✓

Solution B (3 partitions):

Partition 1: [5, 10, 0, 5] → Sum = $20 \geq 15$ ✓ Non-empty: $3 \geq 2$ ✓

Partition 2: [5, 10, 0, 0] → Sum = $15 \geq 15$ ✓ Non-empty: $2 \geq 2$ ✓

Partition 3: [5, 0, 10, 0] → Sum = $15 \geq 15$ ✓ Non-empty: $2 \geq 2$ ✓

Solution C (4 partitions - BETTER!):

Partition 1: [5, 5, 5, 0] → Sum = $15 \geq 15$ ✓ Non-empty: $3 \geq 2$ ✓

Partition 2: [5, 5, 5, 0] → Sum = $15 \geq 15$ ✓ Non-empty: $3 \geq 2$ ✓

Partition 3: [5, 5, 0, 5] → Sum = $15 \geq 15$ ✓ Non-empty: $3 \geq 2$ ✓

Partition 4: [0, 5, 0, 0] → Sum = 5... WAIT, this doesn't work!

Verification of Solution B:

- Index 0: 5 + 5 + 5 = 15 ✓
- Index 1: 10 + 10 + 0 = 20 ✓
- Index 2: 0 + 0 + 10 = 10 ✓
- Index 3: 5 + 0 + 0 = 5 ✓

Example 3: Impossible Case

Original Collection: [3, 2]
Constraints: k = 10, l = 2

Why it's impossible:

- Total sum = 3 + 2 = 5
- We need at least one partition with sum ≥ 10
- But we only have 5 total to distribute!
- This problem has **no valid solution**

Example 4: Tight Diversity Constraint

Original Collection: [20, 0, 0, 0, 0]
Constraints: k = 10, l = 3

Why it's impossible:

- We need each partition to have at least 3 non-empty indices
- But only index 0 has a non-zero value (20)
- We cannot create even one partition with 3 non-empty indices
- This problem has **no valid solution**

Project Requirements

You must implement **TWO algorithms** to solve this problem:

Algorithm 1: Greedy Approach

Design a greedy algorithm that attempts to **maximize the number of partitions** by making locally optimal choices. Your algorithm should:

- Try to create as many valid partitions as possible
- Be efficient (consider time complexity)
- Use appropriate data structures (arrays, heaps, priority queues, etc.)
- Handle the case when no valid solution exists
- Consider strategies like: minimizing the sum/diversity used in each partition to leave room for more partitions

Algorithm 2: Alternative Approach

Design a different algorithm that also **maximizes partition count** using a different strategy. Options include:

- Dynamic programming approach (building up optimal solutions)
- Backtracking with branch-and-bound optimization
- Graph-based formulation with maximum flow concepts
- Integer linear programming formulation
- Heuristic search method with optimization

Compare how effectively each approach maximizes the partition count.

Data Structures to Consider

Your implementation should thoughtfully use data structures such as:

- **Arrays/Lists**: For storing collections and partitions
- **Heaps/Priority Queues**: For efficiently selecting elements
- **Hash Maps/Dictionaries**: For tracking index values and counts
- **Sets**: For managing unique elements or tracking visited states
- **Stacks/Queues**: For backtracking or BFS/DFS approaches

Deliverables

1. Implementation (60%)

- Two working algorithms in Python
- Clear, well-commented code
- Proper use of data structures
- Input validation and error handling

2. Analysis Report (25%)

- Time complexity analysis of both algorithms
- Comparison of the two approaches on partition count maximization
- Discussion of when each algorithm performs better
- Analysis of optimality: Does your algorithm guarantee the maximum number of partitions?
- Test cases showing different scenarios (solvable, unsolvable, edge cases)
- Comparative results showing partition counts achieved by each algorithm

3. Documentation (15%)

- Instructions explaining how to run your code
- Example inputs and outputs
- Explanation of your algorithm design choices
- Discussion of limitations and possible improvements

Testing Your Solution

Your program should:

1. Accept an input collection and constraints k and l
2. Attempt to find valid partitions that **maximize the partition count**
3. Output the partitions if found (showing the count achieved), or indicate no solution exists
4. Verify the solution satisfies all three constraints

Grading Rubric

- **Correctness** (25%): Do your algorithms produce valid partitions?
- **Optimization** (20%): Do your algorithms effectively maximize partition count?
- **Efficiency** (15%): Are your algorithms reasonably efficient?
- **Code Quality** (15%): Is your code clean, readable, and well-structured?
- **Data Structure Usage** (10%): Did you choose and implement appropriate data structures?

- **Analysis** (10%): Is your complexity analysis accurate and thorough?
- **Documentation** (5%): Is your work well-documented and explained?

Bonus Challenges (Optional)

1. **Optimality Proof**: Prove mathematically that your algorithm achieves the maximum possible number of partitions, or provide counterexamples showing it doesn't
2. **Visualization**: Create a visual representation of your partitions showing how they combine to form the original
3. **Scalability**: Test your algorithms on large datasets (n > 1000) and analyze how partition counts scale
4. **Additional Constraints**: Extend the problem with maximum partition size limits while still maximizing count
5. **Approximation Analysis**: If your algorithm doesn't guarantee optimality, analyze how close it gets to the maximum (approximation ratio)

Submission Guidelines

Submit a ZIP file containing:

- algorithm1.py - Your first algorithm implementation
- algorithm2.py - Your second algorithm implementation
- analysis.pdf - Your written analysis report

Due Date: [To be announced]

Good luck!